SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Logic Analysis and Design Challenges

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Logic Analysis and Design Challenges (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q4 – else–if Ladder or switch?

Test results

```
char grade(int marks)
{
   if (marks < 0 || marks > 100) return '?';
   switch (marks / 10) {
      case 10: case 9: return 'S';
      case 8: return 'A';
      case 7: return 'B';
      case 6: return 'C';
      case 5: return 'D';
      default: return 'F';
   }
}
```

Output: 100:S 95:S 83:A 50:D 49:F -5:?

Marks	else–if	switch (guarded)	switch without guard	Comment
100	S	S	S	case 10 needed (100/10)
90 / 89	S / A	S / A	S / A	boundary correct
80 / 79	A / B	A / B	A / B	boundary correct
50 / 49	D / F	D / F	D / F	boundary correct
0	F	F	F	lowest valid
−1	?	?	F (wrong)	−1/10 = 0 (truncation toward zero)
101	?	?	S (wrong)	101/10 = 10 → case 10

Missing break: removing the break after case 8 (so it falls into case 7) changed 83, 89 and 80 from A to B (gcc -Wimplicit-fallthrough warned). Removing a return gives the same fall-through effect: execution continues into the next case.

Ranges: a case label must be one constant integer, so 75–84 would need ten labels; marks / 10 can only express boundaries that are multiples of 10. (case 75 ... 84 is a non-standard GCC extension.)

Easier to change: the ladder – edit one number. The switch must be rewritten, and breaks as soon as a boundary is not a multiple of 10. A lookup table is also easy as long as the boundaries share a step.

Boundary-exposing test set: for every boundary b test b and b − 1: 100, 90, 89, 80, 79, 70, 69, 60, 59, 50, 49, 0; and the invalid neighbours −1, 101; plus extremes INT_MIN and INT_MAX.



Design decision – scholarship rule

```
/* rows: grade S,A,B,C,D,F ; columns: income band 0 (low), 1 (middle), 2 (high) */
static const char *award[6][3] = {
  {"Full","Full","Half"}, {"Full","Half","Quarter"}, {"Half","Quarter","None"},
  {"Quarter","None","None"}, {"None","None","None"}, {"None","None","None"} };
const char *a = award[grade_index][income_band];
```

(The award values are illustrative.) I choose the lookup table: the rule is data, not code, so the board can audit or change it without touching control flow; it is O(1); and a 6 × 3 table is far easier to check than 18 nested if branches. A switch inside an if is acceptable for 2–3 rules but grows quickly with two dimensions; nested if is best only when conditions are not table-shaped (ranges, exceptions).

Alternative Concept and Complexity Analysis

```
static const char grades[11] = {'F','F','F','F','F','D','C','B','A','S','S'};
char g = (m < 0 || m > 100) ? '?' : grades[m / 10];
```

Time: one division and one array read, O(1). The ladder needs up to k comparisons (k = 5 here), O(k); a switch is O(1) when the compiler builds a jump table, O(log k) or O(k) otherwise. Space: 11 bytes for the table (plus the 6 × 3 pointer table for scholarships) versus no data for the ladder. The table has the same weakness as the switch: it needs equal-width ranges.